1

Software Architecture

Foundations

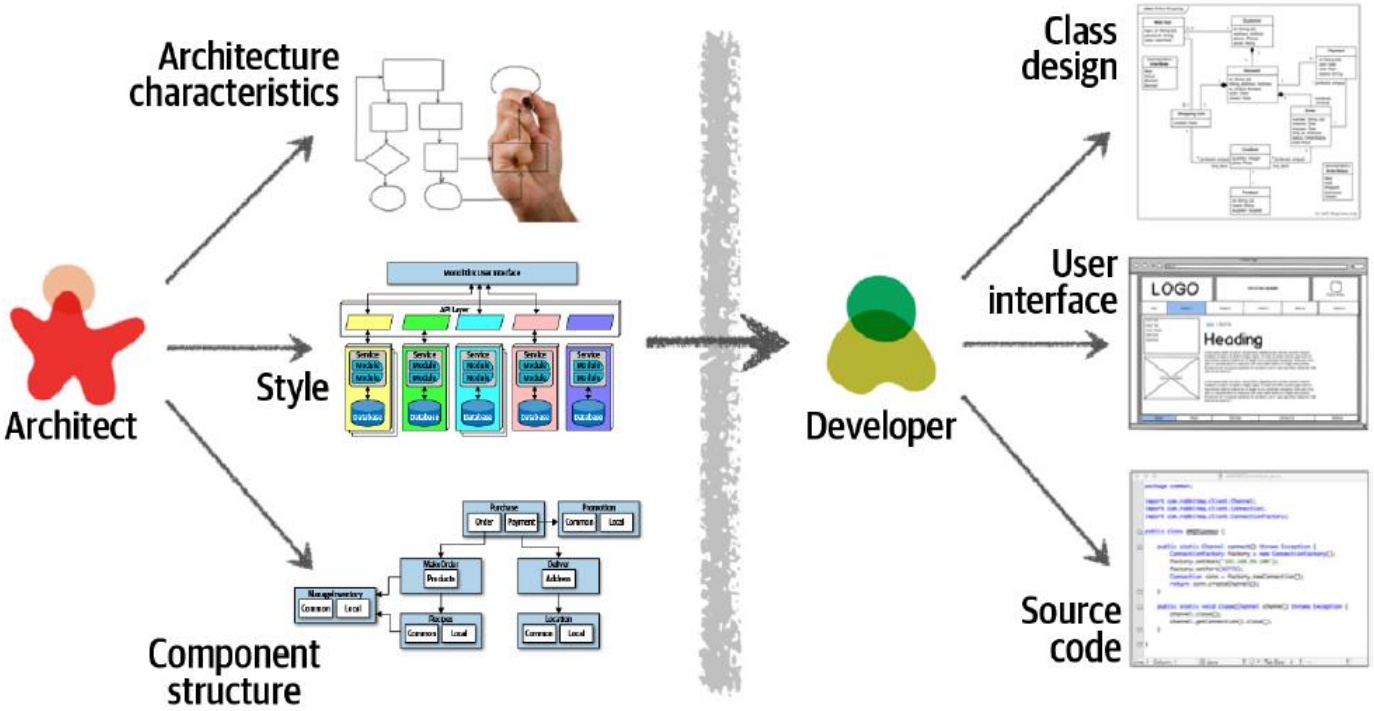
Includes content adapted from *Fundamentals of Software Architecture – An Engineering Approach*, 2020, O'Reilley

1

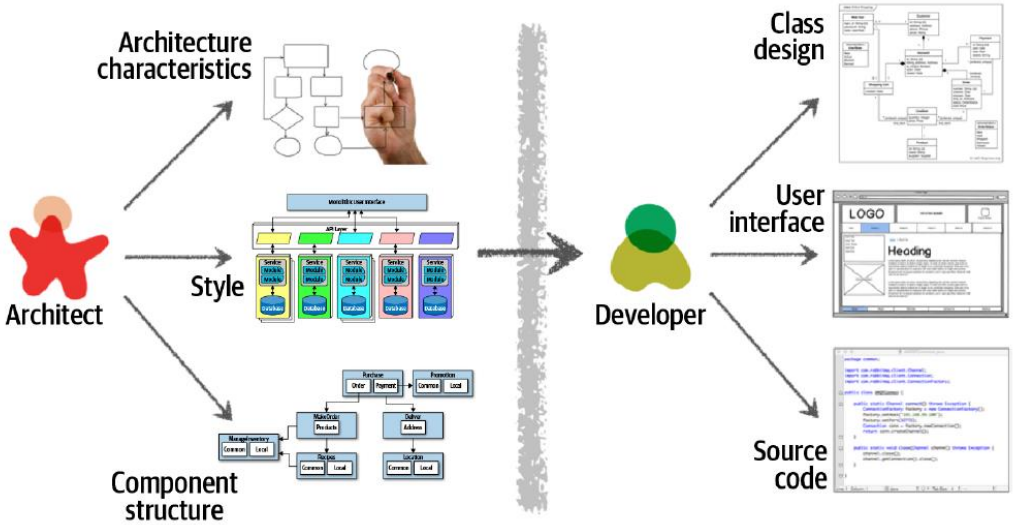
Architectural Thinking

Architecture vs Design and analyzing trade-offs

Traditional view of Architecture vs Design

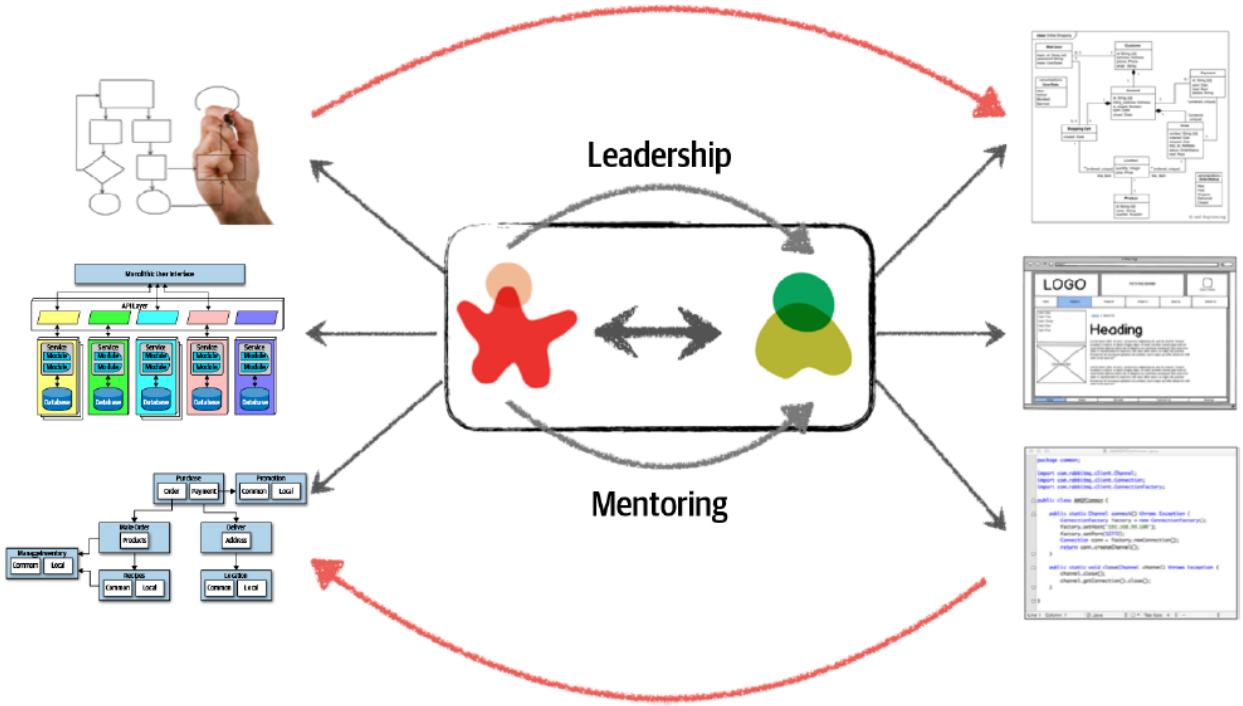


Traditional view of Architecture vs Design



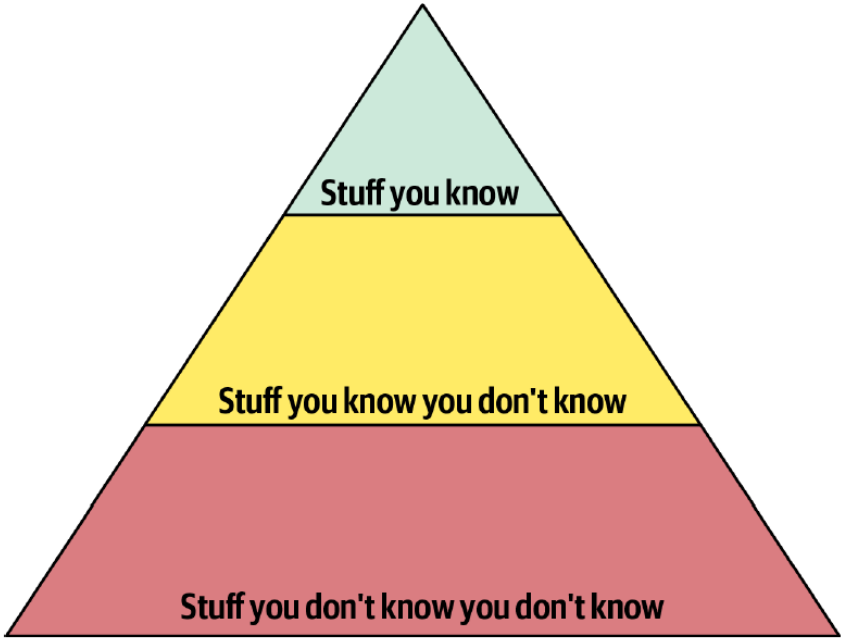
- Architect disconnected from dev team.
- Unidirectional communication
  - *Architect decisions may fail reach developer due to physical and virtual barriers.*
  - *Developer decisions are ignored by architect.*

Collaborative Architecture & Design



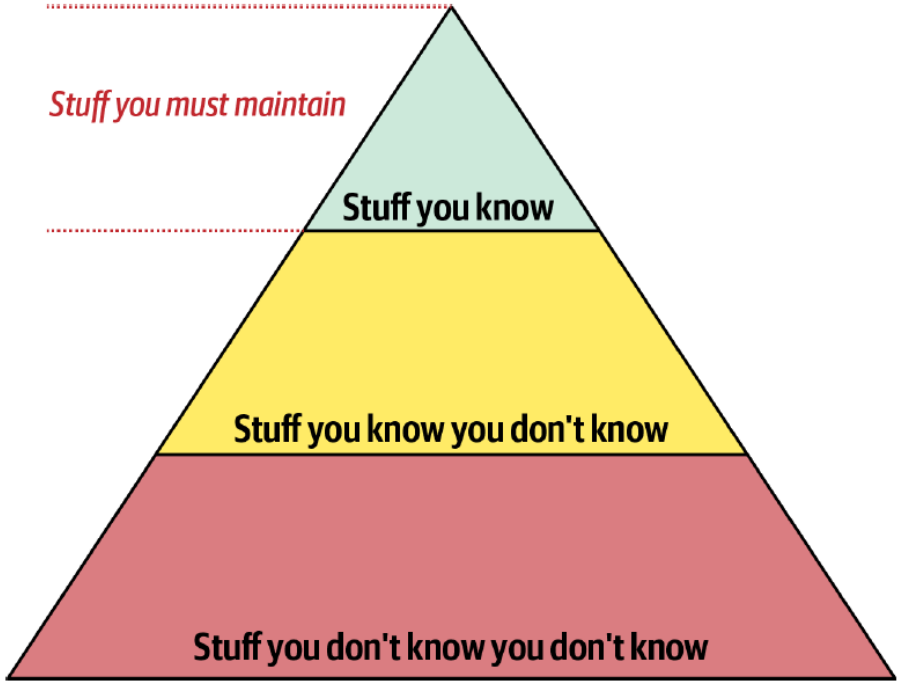
- Tight collaboration between architect and development team.
- Separation between architecture and design is blurred.

Technological Depth vs Technological Breadth



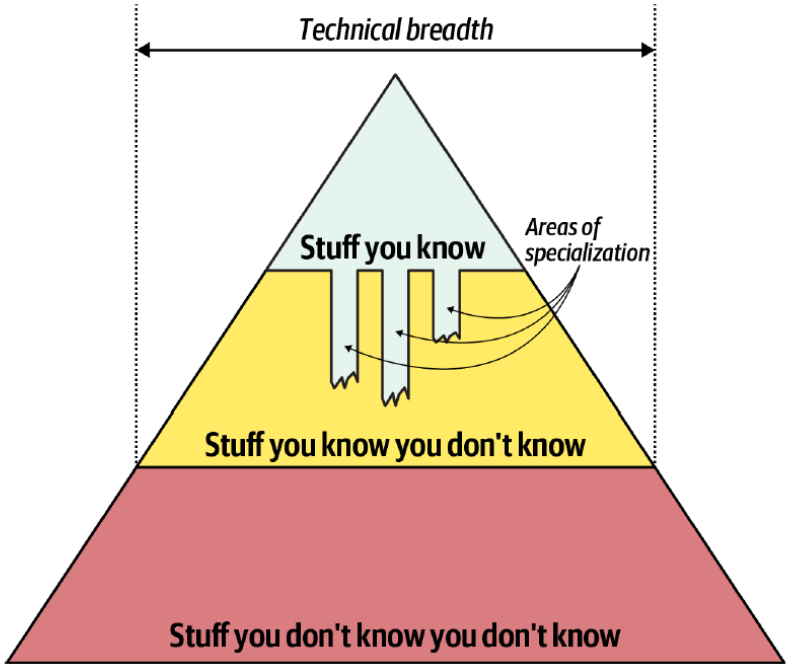
All the knowledge in the world

Technological Depth vs Technological Breadth



- As a developer, you are expected to have a practical, working hands-on knowledge of technology.
- This knowledge must be actively maintained, or it is lost/ becomes obsolete.
- The size of the top pyramid is called the *technical depth.*

Technological Depth vs Technological Breadth



- The role of an architect requires a *broad* understanding of technology and how to use it to solve problems.
  - It is more useful to know about five different technologies that can be used to solve a problem than specializing in only one of them.
  - Both the top AND middle section are relevant.
- The width of the middle section is called the *technical breadth.*



Analyzing Trade-offs

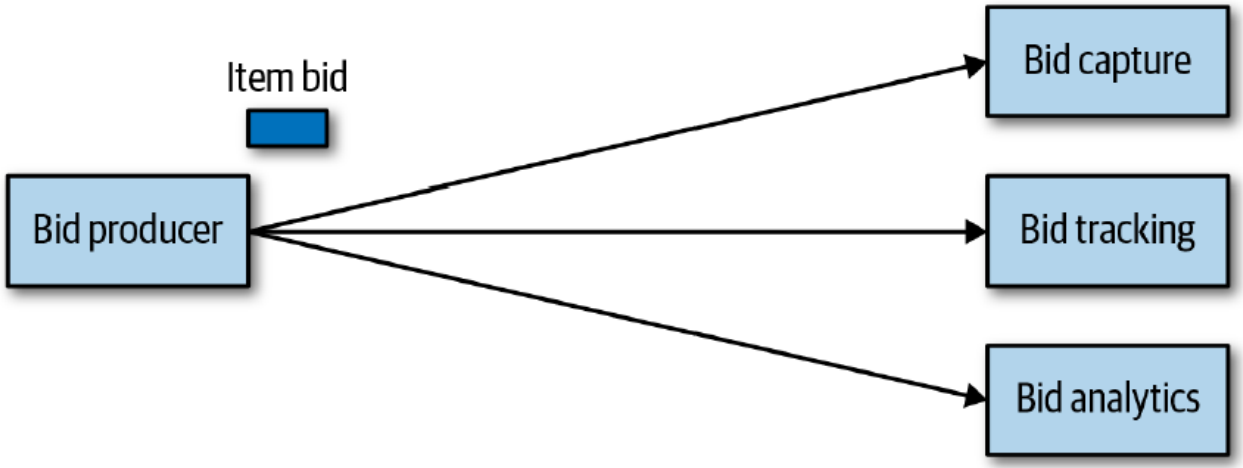
- Thinking like an architect is all about seeing trade-offs in every solution, technical or otherwise, and analyzing those trade-offs to determine what is the best solution.
- The answer to every architectural question is “It depends…”.
- You can’t Google the answer to whether microservices are the best architectural style for *your* system, because *it does depend*…

“Architecture is the stuff you can’t Google!” ***(Mark Richards)***



Trade-Off Analysis Example

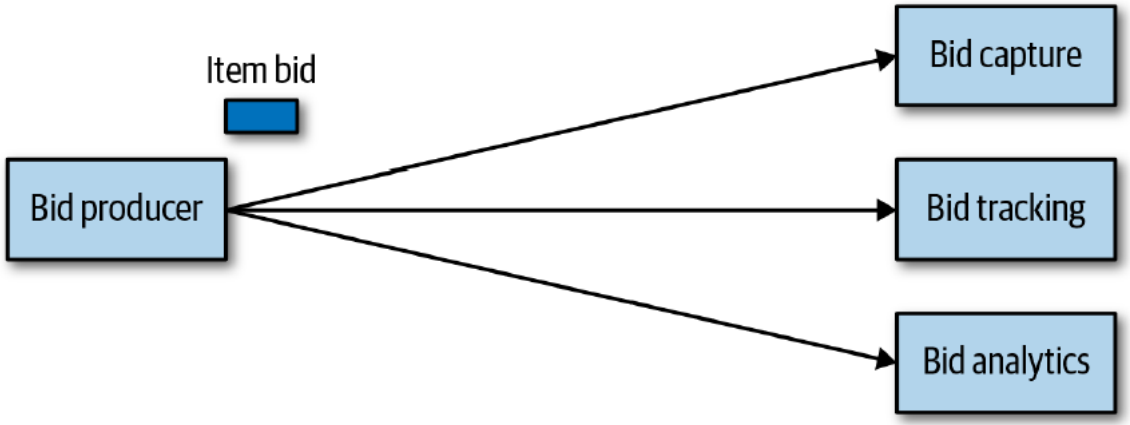
- Consider the following system, where users can place bids for items up for auction.





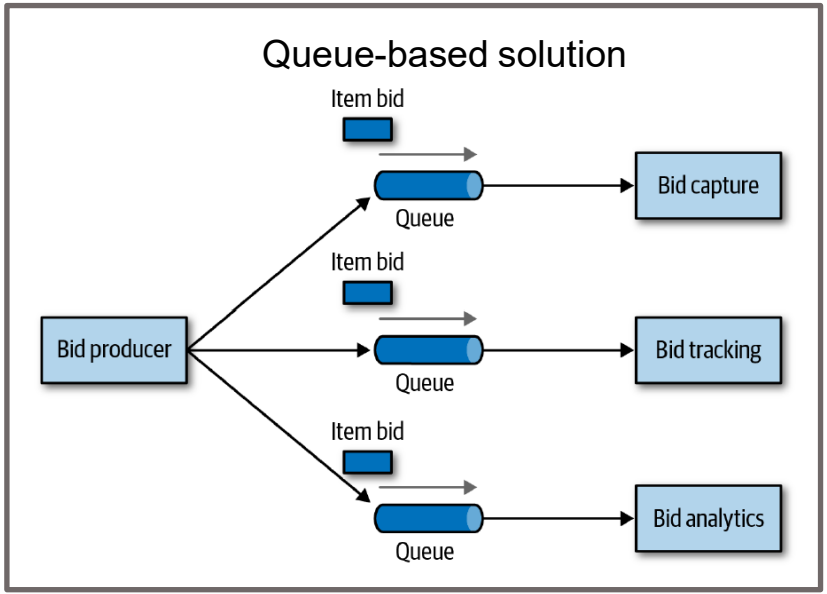
Trade-Off Analysis Example

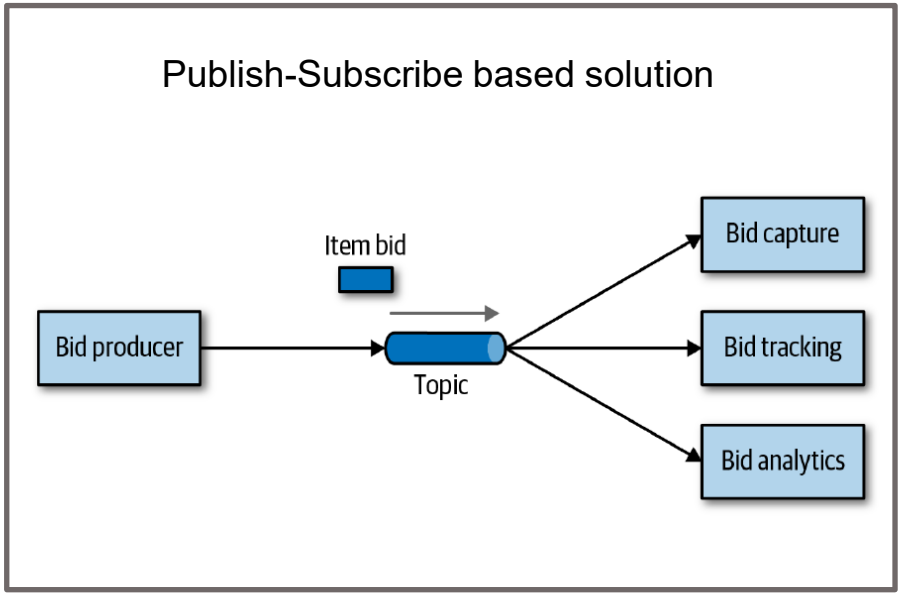
- We want to determine if it is better to use queues and point-to-point messaging, or a topic in a publish-subscribe model.
- *Which option should be used?*



Trade-Off Analysis Example

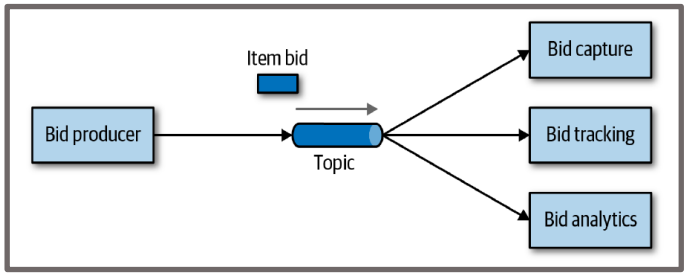
- The two candidate options are the following:

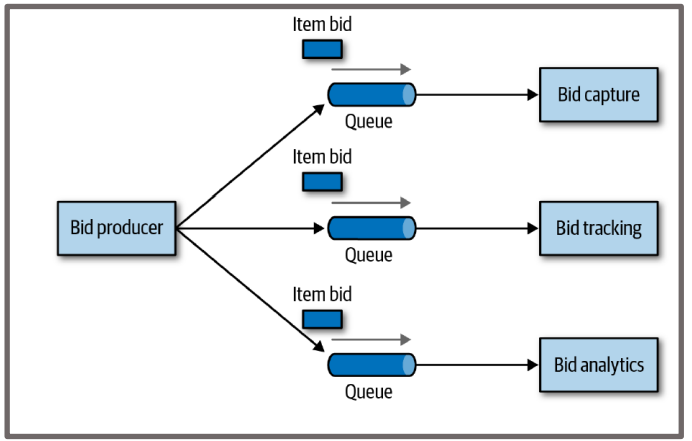




Trade-Off Analysis Example

- The clear advantage of the publish–subscribe model is *architectural extensibility*.
  - The Bid producer connected to single topic.
  - In the other solution, it must connect to ***three*** different queues.
- If a new service becomes necessary:
  - No architectural changes are required in the PS model.
  - In the queue solution, a new queue is necessary, and the producer must be changed to accommodate for the new connection.

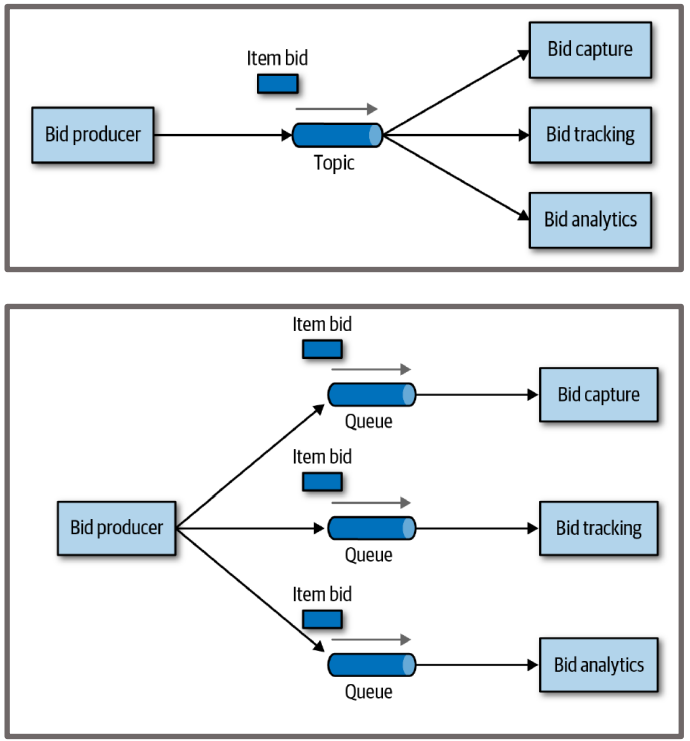




Trade-Off Analysis Example

- Less work may be required when updating the system using the PS model.
- The Bid producer is less coupled with the services.
- In the queue solution, the Bid producer knows exactly which services are using the information, so coupling is higher, and changes are more difficult to implement.

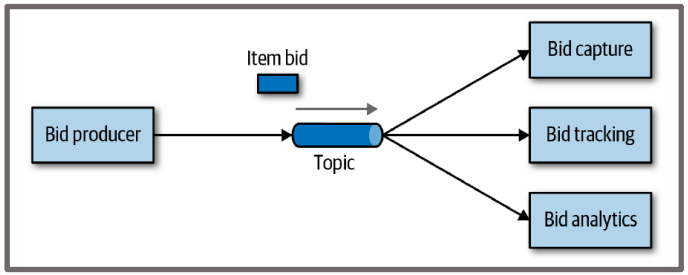
So, we can conclude that the Topic model is clearly the better option in this case.

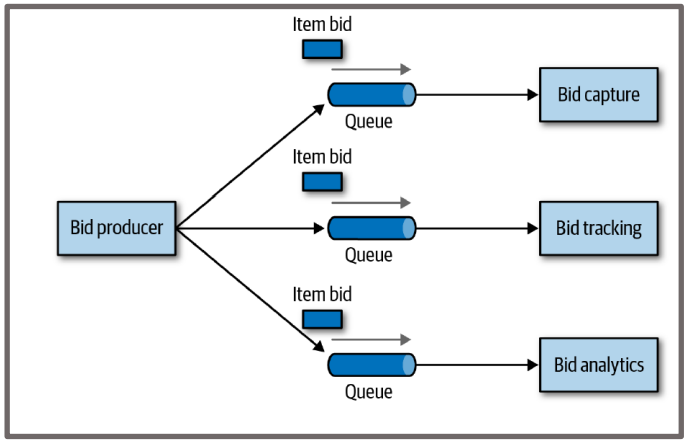


Trade-Off Analysis Example

So, we can conclude that the Topic model is clearly the better option in this case.

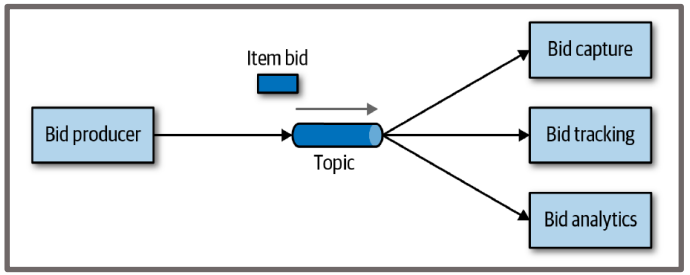
- Is it though?....
- We focused our analysis on the benefits of the Topic solution.
- We need always to consider not only the benefits, but also the *negatives or disadvantages* of a solution.

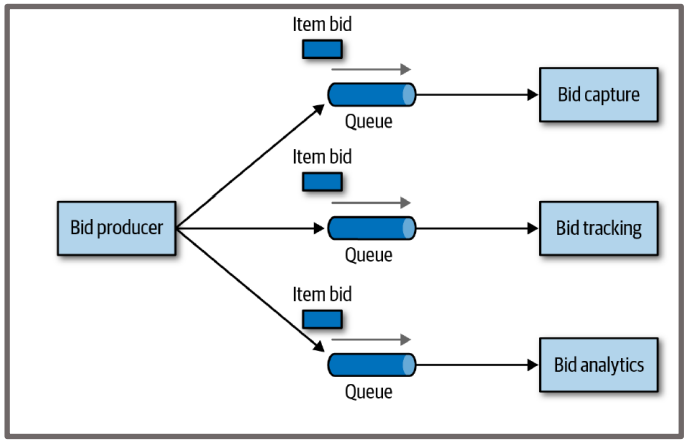




Trade-Off Analysis Example

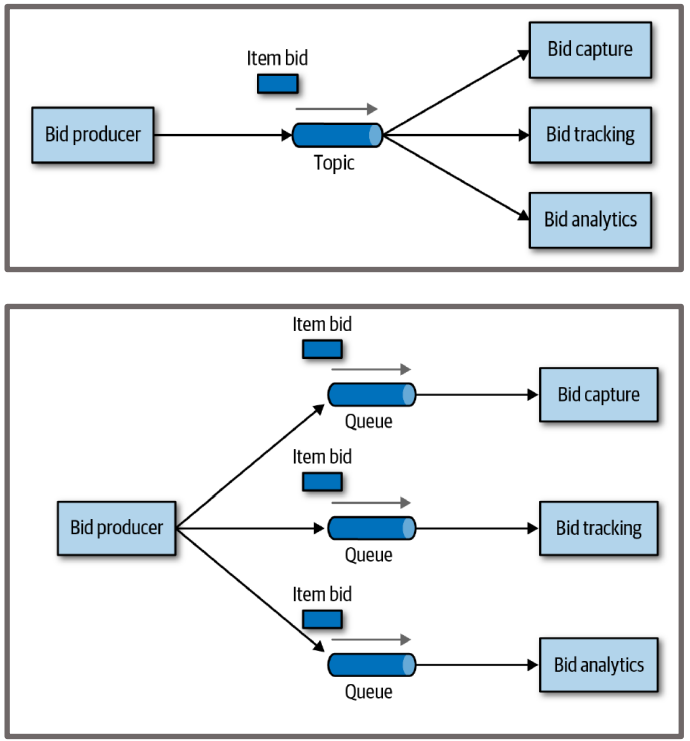
- If we consider data access and data security concerns, we find that:
  - *Any rogue service might access the topic data.*
  - *Data sent to a queue can only be accessed by the specific consumer accessing the message.*
- Unantecipated accesses to queued data prevent delivery to legitimate services.
  - *It is easier to secretly "wiretap" Topics than queues.*





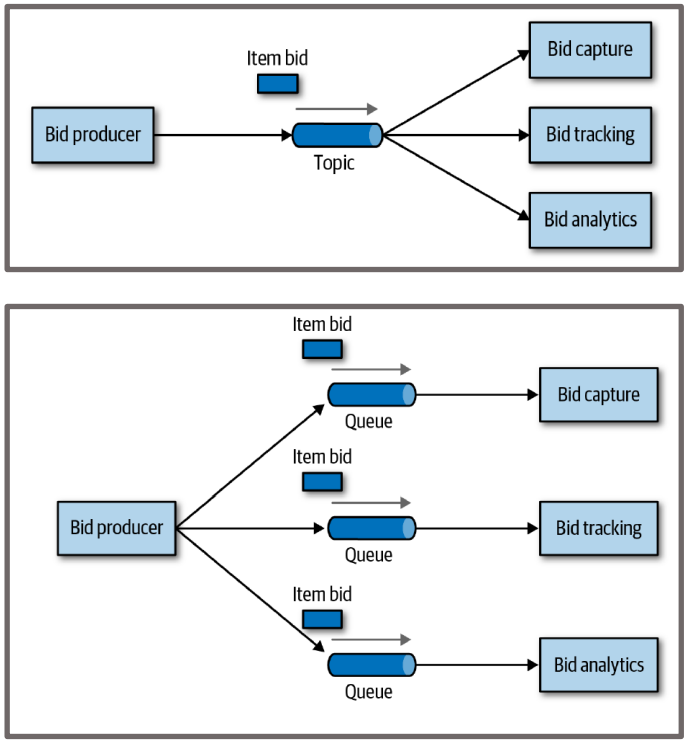
Trade-Off Analysis Example

- **The topic solution requires *homogenous contracts:***
  - *All recipients must accept the same contract and bidding data.*
  - *Accommodating specific data needs of a particular service is not possible.*
  - *This leads to inefficiency (we might need to send more data than is required by any individual service) and difficulty of incorporating new services with specific contract needs (all changes are visible to all Bid consumers).*
  - *Queue models allow different contracts to operate on the different channels*



Trade-Off Analysis Example

- **It is harder to manage auto scaling in the topic solution:**
  - *Independent auto-scaling of consumer services is difficult.*
  - *Queues can be monitored independently, so programmatic load balancing of each independent consumer is possible, separatedly for each service.*





Trade-Off Analysis Example

Topic advantages	Topic disadvantages
Architectural extensibility	Data access and data security concerns
Service decoupling	No heterogeneous contracts
	Monitoring and programmatic scalability

So, having this in mind, we can finally conclude that the best choice...



Trade-Off Analysis Example

Topic advantages	Topic disadvantages
Architectural extensibility	Data access and data security concerns
Service decoupling	No heterogeneous contracts
	Monitoring and programmatic scalability

… depends on what is more important for the system being created.

2

Modularity

How to measure and evaluate modularity



Modularity

- Most platforms provide some means to group related code into *modules*.
- Many definitions of what is a module can be found, with some being mutually contradictory.
- Let's add to the confusion by creating a new definition.

The term *modularity* describes a logical grouping of related code, such as a group of classes in OO programming or a set of functions in a structured or functional language.



Measuring Modularity

- Modularity can be measured with three different concepts or metrics:
  - **Cohesion** – Related to the extent to which different parts of a module should be contained in the same module.
  - **Coupling** – Related to strength of the connections and interdependencies between modules
  - **Connascence** – Related to the level to which changes made to a module will imply changes to another module.



Cohesion

- A measure of how related the parts are to one another.
- High cohesion is good!
- Ideally, all parts in a module should be related to one another.
- Dividing a highly cohesive module will increase coupling.



Cohesion Ranking

- A scale of different categories of cohesion has been described, allowing for comparative analysis of different solutions. These are, from best to worst:
  1. Functional Cohesion
  2. Sequential Cohesion
  3. Communication Cohesion
  4. Procedural Cohesion
  5. Temporal Cohesion
  6. Logical Cohesion
  7. Coincidental Cohesion



Cohesion Ranking

1. **Functional Cohesion** – Every part of the module is related to the other, and the module contains everything essential to function.
2. **Sequential Cohesion** – Two modules interact by having the output of one becoming the input of another.
3. **Communicational Cohesion –** Modules form a communication chain, where each operates on part of the input or contributed somehow to the output result (e.g. add record to DB and generate a new email)



Cohesion Ranking

4. **Procedural Cohesion** – Two modules must execute code in a specific order.
5. **Temporal Cohesion** – Modules are related by timing dependencies *(e.g. initializations before everything else).*
6. **Logical Cohesion –** Data in modules is related logically but not functionally. Operations are related but independent. *(E.g. module converts data to text or a serialized format).*
7. **Coincidental Cohesion** – Modules happen to be in same source file, but are otherwise unrelated.



Cohesion Ranking

- Cohesion is not a very precise metric and is open to interpretation:
- Example: Which of the following is preferrable?

Customer Maintenance
- add customer
- update customer
- get customer
- notify customer
- get customer orders
- cancel customer orders

Customer Maintenance
- add customer
- update customer
- get customer
- notify customer

Order Maintenance
- get customer orders
- cancel customer orders



Cohesion Ranking

- It depends!
  - Are those the only operations in order maintenance? If so, it might make sense to collapse everything into Costumer Maintenance.
  - Is Costumer Maintenance expected to grow much larger?
  - Do the Order operations require much information from the costumer?

Measuring Lack of Cohesion - LCOM

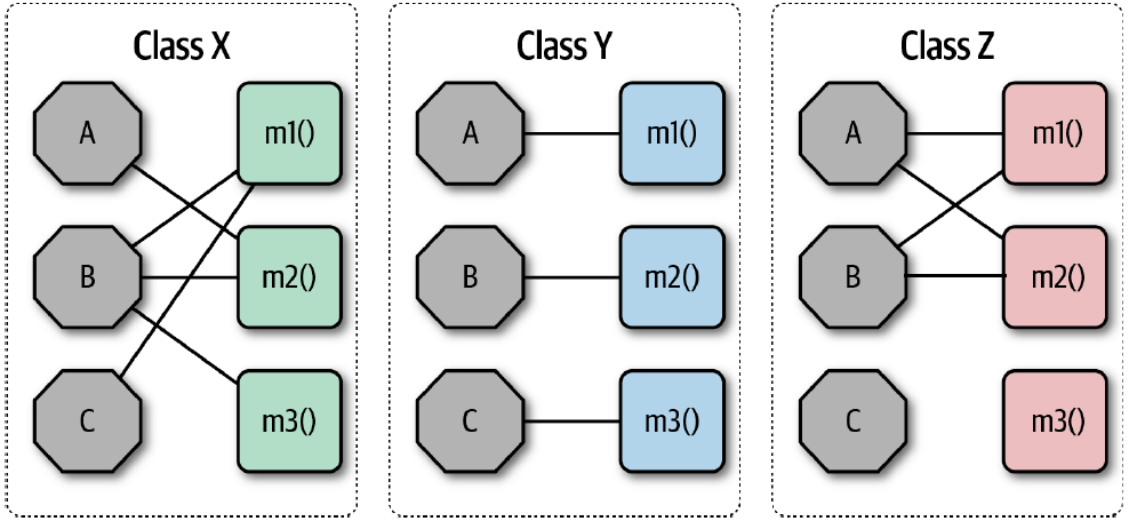
- The Chidamber and Kemerer suite metrics were developed to measure some specific aspects of OO-software.
- The suite includes a LCOM metric (Lack of Cohesion in Methods) that scores the *structural* cohesion of a module. The initial version of LCOM is:

$$LCOM = \begin{cases} |P| - |Q|, & \text{if } |P| > |Q| \\ 0, & \text{otherwise} \end{cases}$$

Where P increases by one for each method that does not access a particular field and Q decreases by one for each method that accesses that field.

Measuring Lack of Cohesion - LCOM

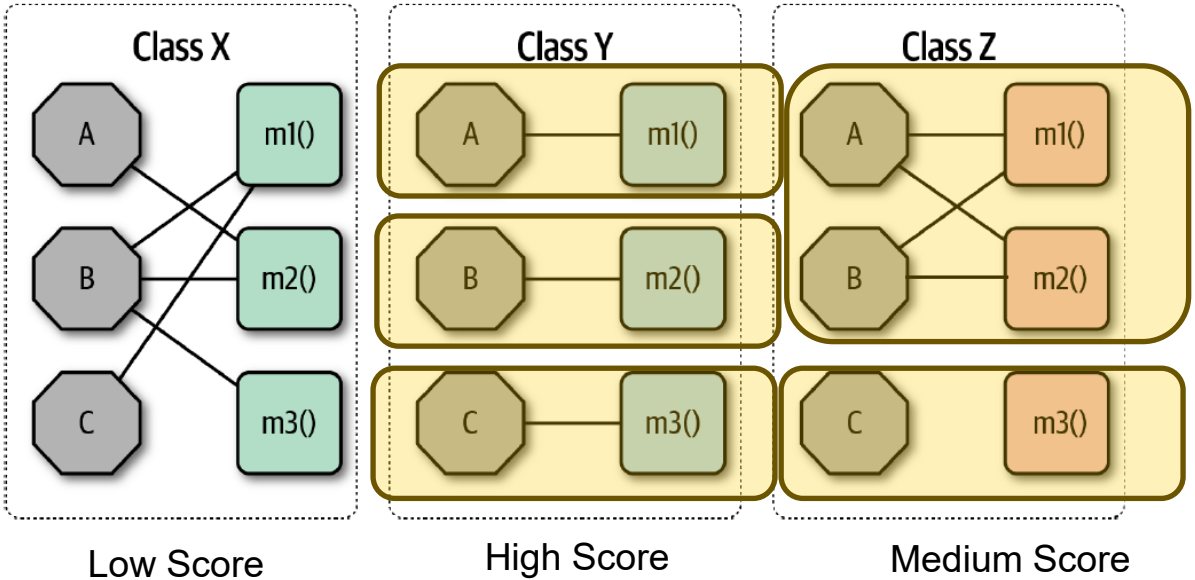
- The initial formula was later updated, and the metric renamed LCOM96b. This metric provides the sum of methods not shared via common fields.



Low Score High Score Medium Score

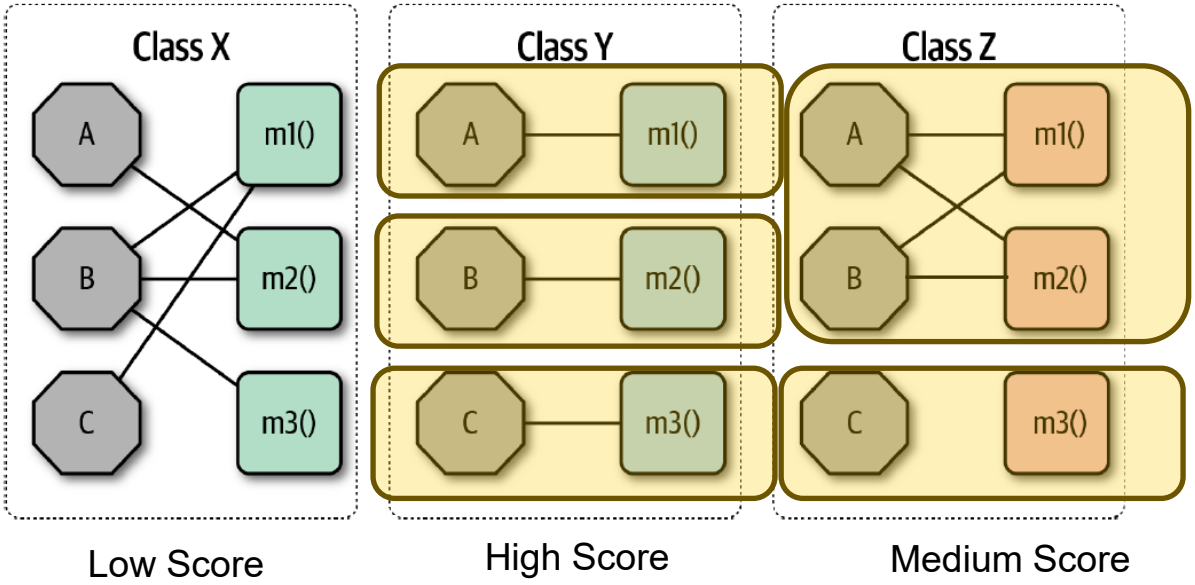
Measuring Lack of Cohesion - LCOM

- The low-score class cannot be refactored without increasing coupling, while the other classes could, to some extent, be refactored into multiple independent classes.



Measuring Lack of Cohesion - LCOM

- Notice that the metrics only address structural Cohesion. Logical cohesion is not measured.





Measuring Lack of Cohesion - LCOM

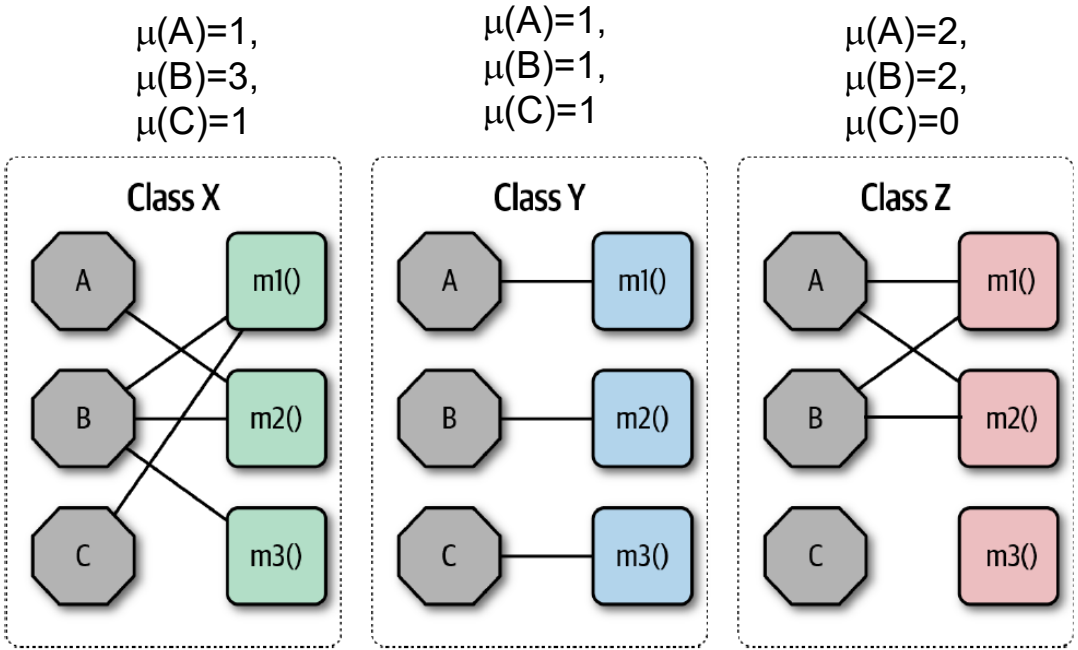
- LCOM96b formula

$$LCOM96b = \frac{1}{a}\sum_{j=1}^{a}\frac{m - \mu(Aj)}{m}$$

Where **a** *is the number of fields,* **m** *the number of methods and* **μ(Aj)** *the number of methods that reference attribute* **Aj**.

Measuring Lack of Cohesion - LCOM

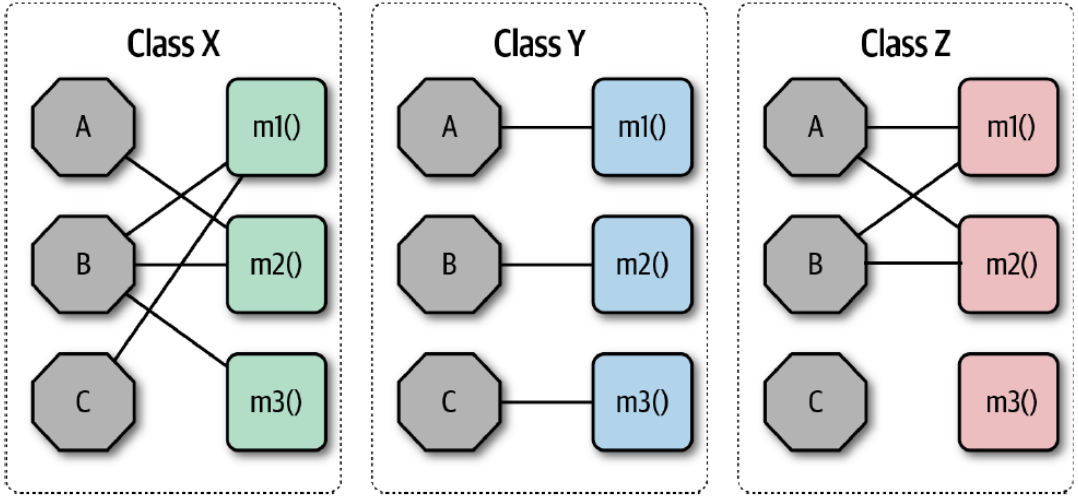
a=3 m=3 for all classes



Measuring Lack of Cohesion - LCOM

a=3 m=3 for all classes

LCOM96b= 0.44 0.66 0.55





Measuring Modularity

- Modularity can be measured with three different concepts or metrics:
  - **Cohesion** – Related to the extent to which different parts of a module should be contained in the same module.
  - **Coupling** – Related to strength of the connections and interdependencies between modules
  - **Connascence** – Related to the level to which changes made to a module will imply changes to another module.



Coupling

- Measures inter module dependencies.
- Measurement is much simpler and objective than cohesion.
- We can classify coupling as:
- **Afferent coupling:** incoming connections.
- **Efferent coupling:** outgoing connections.



Measuring Coupling

- Abstractness, Instability and Distance to main sequence.
  - **Abstractness** – ratio of abstract artifacts (interfaces, abstract classes, etc) to concrete artifacts (implementations).

$$A = \frac{\sum m^a}{\sum m^c}$$

Example: For 1 main function with 5000 lines of code.

A=1/5000



Measuring Coupling

- Abstractness, Instability and Distance to main sequence.
  - **Instability** – ratio of efferent coupling (C^e) to total coupling (C^e+C^a). It determines the volatility of source code.

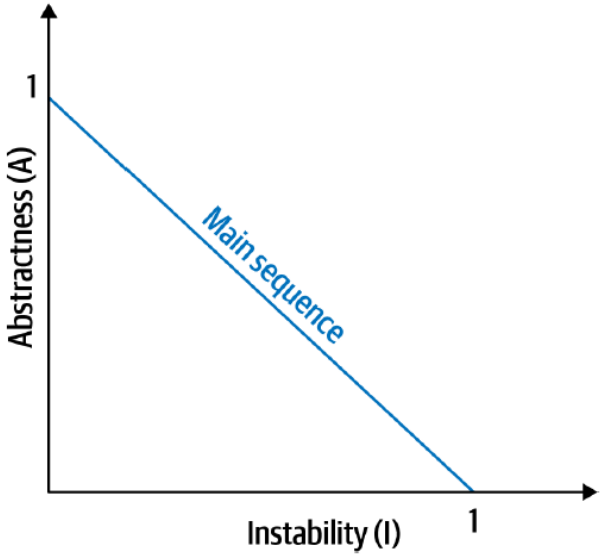
$$I = \frac{C^e}{C^e + C^a}$$

Example: A method that calls 2 methods and is called by one method would have instability of 2/3.



Measuring Coupling

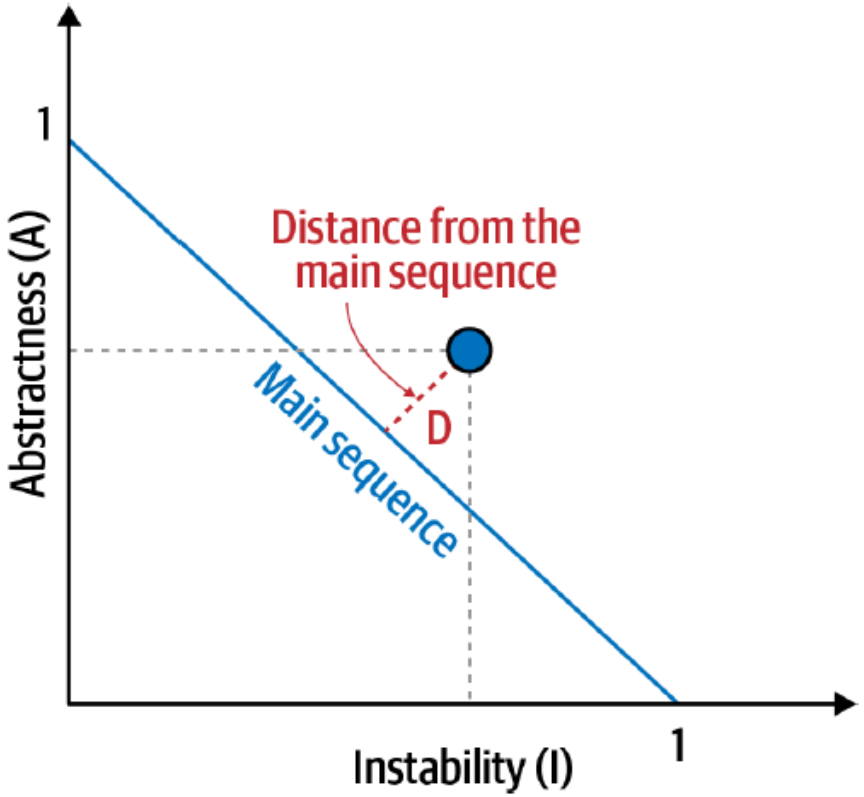
- The maximum value of A and I is 1:
  - *Except for cases of abstractness without practical relevance.*
- The line connecting these two values on a graphic is called the **main sequence**.
- This line is believed to represent a "balanced" mix of abstractness and instability.



Measuring Coupling

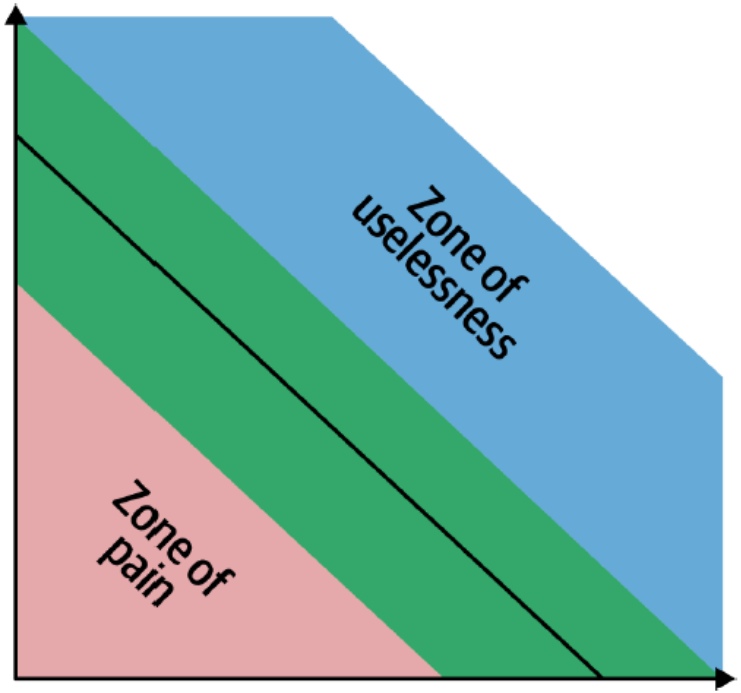
- Abstractness, Instability and Distance to main sequence.
  - **Distance from Main Sequence** – Holistic metric derived from abstractness and instability.

$$D = |A + I - 1|$$



Measuring Coupling

- Having too much or too little abstraction is bad..





Conassance

- Conassance is related to coupling, but further details the nature of the dependencies.
- This allows a more nuanced typification of the dependencies.
- Provides insight into possible refactoring.
- Static Conassance describes source code-level coupling.
- Dynamic Conassance describes behavioral coupling.

Static Connascence

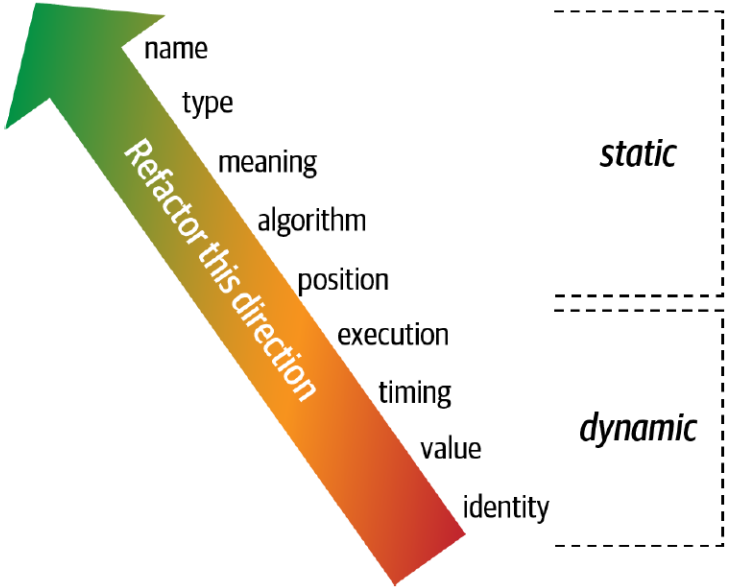
- *Connascence of Name (CoN)*
  - Multiple components must agree on the name of an entity. (e.g name of methods)
- *Connascence of Type (CoT)*
  - Multiple components must agree on the type of an entity.
- *Connascence of Meaning (CoM) or Connascence of Convention (CoC)*
  - Multiple components must agree on the meaning of particular values. (e.g. TRUE=1)
- *Connascence of Position (CoP)*
  - Multiple components must agree on the order of values. (e.g. update(“name”,”address) vs update(“address”,”name”)
- *Connascence of Algorithm (CoA)*
  - Multiple components must agree on a particular algorithm. (e.g. same hash technic on client and server)

Dynamic Connascence

- *Connascence of Execution (CoE)*
  - The order of execution of multiple components is important. (e.g. must set all email properties before sending)
- *Connascence of Timing (CoT)*
  - The timing of the execution of multiple components is important. (e.g. race conditions)
- *Connascence of Values (CoV)*
  - Occurs when several values relate on one another and must change together. (e.g. transactions across multiple databases, constrained values in classes – Rectangle)
- *Connascence of Identity (CoI)*
  - Occurs when multiple components must reference the same entity. (e.g. sharing a distributed queue)

Conassance Properties

- Conassance **Strength** describes the ease of refactoring:
  - Static conassance is preferable to dynamic conassance.
- Conassance **Locality** describes how proximal the modules are i source code.
  - Higher forms of conassance are more acceptable in proximal code.
- Conassance **Degree** describes th size of the impact – many classes or few?



Conassance Properties

- Conassance **Strength** describes the ease of refactoring:
  - Static conassance is preferable to dynamic conassance.
- Conassance **Locality** describes how proximal the modules are in source code.
  - Higher forms of conassance are more acceptable in proximal code.
- Conassance **Degree** describes the size of the impact – many classes or few?

